

126. Passer d'un écran à l'autre

Un titre, une partie, une fin.

1. Ce qu'il faut comprendre

Le squelette de n'importe quel jeu. Deux règles, et il n'y en a pas d'autres.

Une variable dit où l'on est. Un `enum` lui donne ses valeurs possibles, et leur donne des noms : `TITRE`, `JEU`, `FIN`. Le nom de l'`enum` devient un type d'un octet, et `Scene scene` se lit mieux que `uint8_t scene`.

Un seul endroit change d'écran. `allerA()` éteint, efface, redessine, rallume — toujours dans cet ordre. Si chaque bouton faisait son propre ménage, l'un finirait par oublier d'effacer, et deux écrans se mélangeraient.

Le `switch` aiguille sur la scène. Il fonctionne comme en C++, `break` compris : sans lui, l'exécution tombe dans le cas suivant. C'est un piège célèbre, mais c'est le sens du langage, et le trahir en douce serait pire.

On éteint l'écran pour effacer parce que repeindre toute la carte demande bien plus de temps qu'une image n'en contient. `effacer()` sans argument vide les 32×32 de la carte d'un coup — cette leçon écrivait autrefois ses deux boucles à la main, et n'effaçait alors que les 20×18 visibles : le décor qui défilait ramenait l'écran d'avant.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **784 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          52 o ← la console, à l'allumage - la mise en route de la console
allerA()           128 o ← main() - ta fonction
main()             86 o ← démarrage - ta fonction
VBlank             8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage      28 o ← image() - attend l'image suivante
AttendreVBlank     16 o ← ecran() - attend que l'écran soit libre
AttendreAcces      12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte        10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles       76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage - vide les deux cartes de l'écran
EffacerFond        17 o ← effacer() - vide tout le fond
LireManette        37 o ← bouton() - lit les huit boutons
Tuiles            240 o ← CopierTuiles - 15 lettres sur 43
"MON JEU"          7 o  ← texte() - un texte gravé
"A POUR JOUER"     12 o ← texte() - un texte gravé
```

```

"C EST PARTI"      11 o ← texte() – un texte gravé
"A POUR FINIR"     12 o ← texte() – un texte gravé
"FINI"             4 o  ← texte() – un texte gravé
"A POUR REVENIR"   14 o ← texte() – un texte gravé

```

Lettres gravées : ACEFIJMNOPRSTUV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerPanneau,

DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte()   × 6    66 o sur place, dans allerA() + EcrireTexte (10 o)
ecran()   × 2    21 o sur place, dans allerA() + AttendreVBlank (16 o)
bouton()  × 1    12 o sur place, dans main() + LireManette (37 o)
effacer() × 1     3 o sur place, dans allerA() + EffacerFond (17 o)
image()   × 1     3 o sur place, dans main() + AttendreImage (28 o)

```

Tes fonctions appelées :

```

allerA() × 4    26 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <ecran>      // éteint ou rallume l'écran
#include <effacer>    // efface des cases, ou tout le fond
#include <texte>      // écrit un texte à l'écran
#include <bouton>     // lit un bouton de la manette

```

2. DES CONSTANTES NOMMÉES — SCENE

```

enum Scene { TITRE, JEU, FIN };

```

3. UNE VARIABLE — SCENE

```

Scene scene = TITRE;

```

4. UNE FONCTION — ALLERA()

```

void allerA(Scene ou) {
    scene = ou;

    ecran(0);
    effacer();

    switch (scene) {
        case TITRE:
            texte(6, 5, "MON JEU");
            texte(4, 10, "A POUR JOUER");
            break;

        case JEU:
            texte(5, 4, "C EST PARTI");
            texte(4, 10, "A POUR FINIR");
            break;

        case FIN:
            texte(7, 5, "FINI");
            texte(3, 10, "A POUR REVENIR");
            break;
    }

    ecran(1);
}

```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t aAvant = 0;

    allerA(TITRE);

    while (true) {
        image();

        uint8_t a = bouton(A);

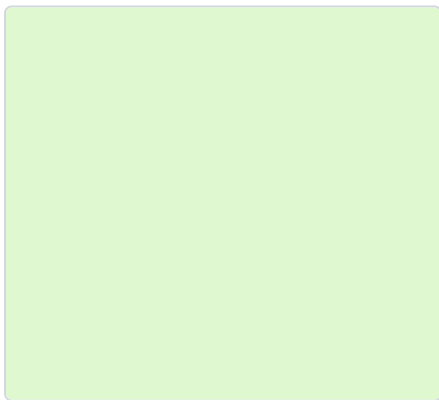
        if (a && !aAvant) {
            if (scene == TITRE) allerA(JEU);
            else if (scene == JEU) allerA(FIN);
            else allerA(TITRE);
        }

        aAvant = a;
    }

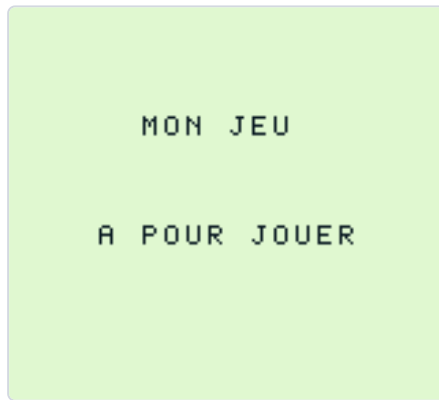
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

C EST PARTI

A POUR FINIR

A — l'écran a changé

5. Ce qu'on doit voir

« MON JEU », puis A change d'écran à chaque appui.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ on démarre sur le titre
- ✓ A mène à la partie
- ✓ puis à la fin
- ✓ et on revient au titre
- ✓ l'ancien écran est effacé

Ensuite : **127. La pesanteur et le saut** — chapitre 23 : Un vrai jeu. · gameboy3 — écrire une cartouche Game Boy en C++